

PostgreSQL SQL Commands — DDL, DML, DCL

Using PostgreSQL and AdventureWorks

1. What is DDL, DML, DCL?

Type	Full Form	Purpose
DDL	Data Definition Language	Defines database structure
DML	Data Manipulation Language	Manipulates data
DCL	Data Control Language	Controls permissions/security

2. DDL — Data Definition Language

DDL commands are used to:

- Create objects
- Modify objects
- Delete objects
- Define schema structure

Main DDL Commands:

CREATE
ALTER
DROP
TRUNCATE
RENAME

AdventureWorks Tables Used

We will use tables similar to AdventureWorks:

- Person.Person
 - HumanResources.Employee
 - Sales.SalesOrderHeader
 - Production.Product
-

3. CREATE TABLE

Creates a new table.

Example

```
CREATE TABLE HumanResources.Employee_Test
(  
    EmployeeID INT PRIMARY KEY,  
    FirstName VARCHAR(100),  
    LastName VARCHAR(100),  
    Department VARCHAR(50),  
    Salary NUMERIC(10,2),  
    JoiningDate DATE  
);
```

Explanation

Line	Meaning
CREATE TABLE	Creates table
Employee_Test	Table name
PRIMARY KEY	Unique identifier
VARCHAR	Variable length string
NUMERIC(10,2)	Decimal values
DATE	Date datatype

4. CREATE TABLE FROM EXISTING TABLE

```
CREATE TABLE HumanResources.Employee_Backup AS  
SELECT *  
FROM HumanResources.Employee;
```

Explanation

Creates backup table with existing data.

5. ALTER TABLE

Used to modify existing table structure.

ADD COLUMN

```
ALTER TABLE HumanResources.Employee_Test  
ADD COLUMN Email VARCHAR(200);
```

MODIFY COLUMN

(PostgreSQL uses ALTER COLUMN TYPE)

```
ALTER TABLE HumanResources.Employee_Test  
ALTER COLUMN Department TYPE VARCHAR(100);
```

RENAME COLUMN

```
ALTER TABLE HumanResources.Employee_Test
```

```
RENAME COLUMN Salary TO MonthlySalary;
```

DROP COLUMN

```
ALTER TABLE HumanResources.Employee_Test  
DROP COLUMN Email;
```

6. DROP TABLE

Deletes entire table permanently.

```
DROP TABLE HumanResources.Employee_Test;
```

7. TRUNCATE TABLE

Deletes all rows but keeps structure.

```
TRUNCATE TABLE HumanResources.Employee_Test;
```

Difference: DELETE vs TRUNCATE

DELETE	TRUNCATE
Removes rows one by one	Removes all rows instantly
Can use WHERE	Cannot use WHERE
Slower	Faster
Logged row-by-row	Minimal logging

8. RENAME TABLE

```
ALTER TABLE HumanResources.Employee_Test  
RENAME TO Employee_Stage;
```

9. CREATE SCHEMA

```
CREATE SCHEMA Training;
```

10. CREATE VIEW

```
CREATE VIEW vw_EmployeeDetails AS  
SELECT  
    EmployeeID,  
    FirstName,  
    LastName,  
    Department  
FROM HumanResources.Employee_Test;
```

Explanation

View acts like virtual table.

11. DML — Data Manipulation Language

DML commands manipulate data.

Main Commands:

```
INSERT  
UPDATE  
DELETE  
SELECT  
MERGE
```

12. INSERT

Adds new records.

Insert Single Row

```
INSERT INTO HumanResources.Employee_Test
(
    EmployeeID,
    FirstName,
    LastName,
    Department,
    Salary,
    JoiningDate
)
VALUES
(
    1,
    'Raj',
    'Kumar',
    'IT',
    75000,
    '2025-01-15'
);
```

Insert Multiple Rows

```
INSERT INTO HumanResources.Employee_Test
VALUES
(2,'Anu','Priya','HR',65000,'2024-05-01'),
(3,'Vijay','M','Finance',82000,'2023-08-11'),
(4,'Arun','K','IT',91000,'2022-09-20');
```

13. SELECT

Retrieves data.

Basic SELECT

```
SELECT *  
FROM HumanResources.Employee_Test;
```

Specific Columns

```
SELECT  
    FirstName,  
    LastName,  
    Salary  
FROM HumanResources.Employee_Test;
```

WHERE Condition

```
SELECT *  
FROM HumanResources.Employee_Test  
WHERE Department = 'IT';
```

ORDER BY

```
SELECT *  
FROM HumanResources.Employee_Test  
ORDER BY Salary DESC;
```

DISTINCT

```
SELECT DISTINCT Department  
FROM HumanResources.Employee_Test;
```

LIMIT

```
SELECT *  
FROM HumanResources.Employee_Test  
LIMIT 5;
```

14. UPDATE

Modifies existing rows.

Update Single Row

```
UPDATE HumanResources.Employee_Test  
SET Salary = 95000  
WHERE EmployeeID = 1;
```

Update Multiple Columns

```
UPDATE HumanResources.Employee_Test  
SET  
    Department = 'Data Engineering',  
    Salary = 120000  
WHERE EmployeeID = 3;
```

Important

Always use WHERE condition.

Without WHERE:

```
UPDATE HumanResources.Employee_Test  
SET Salary = 50000;
```

This updates ALL rows.

15. DELETE

Deletes rows.

Delete Specific Row

```
DELETE FROM HumanResources.Employee_Test  
WHERE EmployeeID = 4;
```

Delete All Rows

```
DELETE FROM HumanResources.Employee_Test;
```

16. MERGE (UPSERT in PostgreSQL)

PostgreSQL uses:

```
INSERT ... ON CONFLICT
```

Example

```
INSERT INTO HumanResources.Employee_Test  
(  
    EmployeeID,  
    FirstName,  
    LastName,  
    Department,  
    Salary  
)  
VALUES  
(
```

```
1,  
'Raj',  
'Kumar',  
'IT',  
85000  
)  
ON CONFLICT (EmployeeID)  
DO UPDATE  
SET Salary = EXCLUDED.Salary;
```

Explanation

Part	Meaning
ON CONFLICT	Checks duplicate key
DO UPDATE	Updates existing row
EXCLUDED	New incoming value

17. DCL — Data Control Language

Controls user access and permissions.

Main Commands:

```
GRANT  
REVOKE
```

18. CREATE USER

```
CREATE USER trainee_user  
WITH PASSWORD 'StrongPassword123';
```

19. GRANT Permissions

Grant SELECT

```
GRANT SELECT  
ON HumanResources.Employee_Test  
TO trainee_user;
```

Grant INSERT, UPDATE

```
GRANT INSERT, UPDATE  
ON HumanResources.Employee_Test  
TO trainee_user;
```

Grant All Privileges

```
GRANT ALL PRIVILEGES  
ON HumanResources.Employee_Test  
TO trainee_user;
```

Grant Schema Usage

```
GRANT USAGE  
ON SCHEMA HumanResources  
TO trainee_user;
```

20. REVOKE Permissions

Revoke SELECT

```
REVOKE SELECT  
ON HumanResources.Employee_Test  
FROM trainee_user;
```

Revoke All

```
REVOKE ALL PRIVILEGES  
ON HumanResources.Employee_Test  
FROM trainee_user;
```

21. Complete End-to-End Example

Step 1 — Create Table

```
CREATE TABLE Sales.Customer_Test  
(  
    CustomerID INT PRIMARY KEY,  
    CustomerName VARCHAR(100),  
    City VARCHAR(100),  
    CreditLimit NUMERIC(10,2)  
);
```

Step 2 — Insert Data

```
INSERT INTO Sales.Customer_Test  
VALUES  
(1,'Rajesh','Chennai',100000),  
(2,'Anu','Bangalore',75000),  
(3,'Karthik','Hyderabad',120000);
```

Step 3 — Read Data

```
SELECT *  
FROM Sales.Customer_Test;
```

Step 4 — Update Data

```
UPDATE Sales.Customer_Test  
SET CreditLimit = 150000  
WHERE CustomerID = 1;
```

Step 5 — Delete Data

```
DELETE FROM Sales.Customer_Test  
WHERE CustomerID = 2;
```

Step 6 — Grant Access

```
GRANT SELECT, INSERT  
ON Sales.Customer_Test  
TO trainee_user;
```

Step 7 — Revoke Access

```
REVOKE INSERT  
ON Sales.Customer_Test  
FROM trainee_user;
```

22. Real AdventureWorks Queries

Products Above Average Price

```
SELECT
    Name,
    ListPrice
FROM Production.Product
WHERE ListPrice >
(
    SELECT AVG(ListPrice)
    FROM Production.Product
);
```

Employees in Specific Department

```
SELECT
    e.BusinessEntityID,
    p.FirstName,
    p.LastName
FROM HumanResources.Employee e
JOIN Person.Person p
ON e.BusinessEntityID = p.BusinessEntityID;
```

Sales Orders Count

```
SELECT
    CustomerID,
    COUNT(*) AS TotalOrders
FROM Sales.SalesOrderHeader
GROUP BY CustomerID
ORDER BY TotalOrders DESC;
```

23. Important Interview Differences

Feature	DDL	DML	DCL
Purpose	Structure	Data	Permissions

Auto Commit	Usually Yes	No	Yes
Rollback Possible	Usually No	Yes	Usually No
Examples	CREATE	INSERT	GRANT

24. PostgreSQL Useful Commands (Terminal)

Connect:

```
psql -U postgres
```

List databases:

```
\l
```

Connect database:

```
\c AdventureWorks
```

List tables:

```
\dt
```

Describe table:

```
\d HumanResources.Employee
```

Quit:

```
\q
```